Name: _____ Reg #: _____ Section: _____

National University of Computer and Emerging Sciences, Lahore Campus

Course:	Design and Analysis of Algorithms	Course Code:	CS2009
Program:	BS(Data Science)	Semester:	Fall 2023
Duration:	30 Minutes	Total Marks:	11
Paper Date:	31-Oct-2023	Weight	%
Section:	5B	Page(s):	4
Exam:	Quiz 3		

Q1) The semester is over! You've rented a car and are ready to set out on a long drive on the Strange Highway. There are n tourist stores on the Strange Highway, numbered 1, 2, …, n and you would want to stop at these and buy some souvenirs (only one souvenir may be bought from a store and all souvenirs everywhere have the same price). You are a greedy shopper and are most interested in maximizing the total discount you get on your shopping. You know that each store i offers a discount d_i on a souvenir. But it's a strange highway after all. It turns out that you can only buy a souvenir from a store i if you have not bought anything from the previous f_i stores. For example, if $f_6 = 3$ then you can only buy a souvenir from store 6, and get the discount d_6, if you haven't bought anything from stores 3, 4, and 5. All the d_i and f_i are known to you in advance (passed as input). You have recently learnt the DP technique in your algorithms course and wish to apply it here in order to maximize your total discount under the given constraints. I will help you by defining the optimal sub-structure. [6 Marks]

$D[i]$ = max total discount when store i is the last store where you buy a souvenir.

Example

i	f(i)	d(i)	D(i)
1	0	2	2
2	0	4	6
3	2	2	2
4	2	5	7

(a) Provide recurrences for D[i].

$$D[i] = \max_{1<k<(i-f_i)} \{D[k]\} + d_i$$

(b) Provide complete pseudo-code of the bottom-up DP algorithm based on the recurrence above.

Solution:

```
int Function( char *d, char *f, int n )
{
   D[1] = d[1]
   for (i = 2 to n)
            max = -1
            for (k = 1 to (i-f[i]))
                  if (D[k] > max)
                        max = D[k]
            D[i] = max +d[i]

  max = -1
  for (k = n to (n-f[n]))
       if (D[k] > max)
            max = D[k]
  return max
}
```

Name: _____ Reg #: _____ Section: _____

Q2) Dry run longest common subsequence (LCS) dynamic programming algorithm on the following sequences:

X = CCTAT

Y = TACGC

Fill the following table. Each cell contains length of LCS of a subproblem. Place arrows pointing to the previous cell which is used for defining values of current cell. [1 + 4 = 5 Marks], 1 mark for writing correct values, 4 marks for placing correct arrows.

X Y	C	C	T	A	T
T					
A					
C					
G					
C					
A					
T					

LCS Algorithm

LCS-Length(X, Y)

1. m = length(X) // get the # of symbols in X

2. n = length(Y) // get the # of symbols in Y

3. for i = 1 to m c[i,0] = 0 // special case: Y_0

4. for j = 1 to n c[0,j] = 0 // special case: X_0

5. for i = 1 to m // for all X_i

6. for j = 1 to n // for all Y_j

7. if (X_i == Y_j)

8. c[i,j] = c[i-1,j-1] + 1

9. else

10. c[i,j] = max(c[i-1,j], c[i,j-1])

11. return c